Next Word Prediction Project Guide

1. Project Introduction

This project is a full-stack next word prediction system. Its goal is to help users type faster by suggesting the most likely next words based on the text they have already entered.

The project contains three main parts:

- a frontend where the user types text and sees suggestions
- a backend API that receives the text and returns predictions
- a machine learning training pipeline that prepares and trains the LSTM model

This guide explains the project from start to end in simple language.

2. Main Idea Of The Project

The idea is very simple:

1. The user types a few words.
2. The frontend sends that text to the backend.
3. The backend checks whether a trained LSTM model is available.
4. If the model is available, it predicts the top next words.
5. If the model is not available, the backend uses the fallback predictor.
6. The frontend shows the suggestions as buttons.
7. When the user clicks one, it gets added to the sentence and new suggestions are generated.

So the project behaves like a small smart keyboard suggestion system.

3. Project Flow From Start To End

The complete flow is:

- dataset text is stored in the data folder
- the training script reads the dataset
- the text is cleaned and tokenized
- sequences are created for next-word learning
- the LSTM model is trained
- the tokenizer and trained model are saved in artifacts
- the Flask backend loads those artifacts during runtime
- the frontend sends user text to the backend API
- the backend returns prediction results in JSON
- the frontend renders those words on the screen

4. Folder Structure

Important folders and files used in this project:

- ‘app.py‘
- ‘backend/‘
- ‘frontend/‘
- ‘training/‘
- ‘data/‘
- ‘artifacts/‘
- ‘tests/‘
- ‘README.md‘
- ‘Dockerfile‘

What each one does:

- ‘app.py‘: starts the Flask server and connects routes to the predictor
- ‘backend/‘: contains prediction logic and helper code
- ‘frontend/‘: contains the user interface files
- ‘training/‘: contains the model training script
- ‘data/‘: contains the dataset text file
- ‘artifacts/‘: stores the trained model, tokenizer, metadata, and metrics
- ‘tests/‘: stores the smoke test used to quickly verify the app
- ‘README.md‘: setup and deployment instructions
- ‘Dockerfile‘: used for container deployment

5. Frontend Tools Used

HTML

Why it is used:

- HTML builds the structure of the web page
- it defines the heading, text area, buttons, and suggestion area

How it works here:

- the user types inside a textarea
- buttons are used for quick prompts, clearing text, copying text, and reloading model status
- a suggestion area shows next-word options

File used:

- ‘frontend/index.html‘

CSS

Why it is used:

- CSS styles the page and makes it look good
- it improves spacing, color, layout, and mobile responsiveness

How it works here:

- it creates the card design
- it styles the heading section
- it styles buttons, suggestion blocks, and status pills
- it helps the page work on desktop and mobile

File used:

- ‘frontend/style.css‘

JavaScript

Why it is used:

- JavaScript adds interactivity to the frontend
- it sends data to the backend without reloading the page

How it works here:

- reads the text typed by the user
- sends it to ‘/api/predict‘
- receives prediction results
- displays suggestions
- appends the chosen suggestion to the sentence
- updates status and word count

File used:

- ‘frontend/script.js‘

6. Backend Tools Used

Python

Why it is used:

- Python is easy to read and is strong for machine learning and backend work

How it works here:

- Flask routes are written in Python

- model loading and prediction logic are written in Python
- the training script is also written in Python

Flask

Why it is used:

- Flask is a lightweight web framework
- it is perfect for building small to medium backend APIs

How it works here:

- serves the frontend page
- provides the prediction API
- provides health and project-info routes
- provides a reload route for model status refresh

Main route files:

- ‘app.py‘

Routes used:

- ‘/‘
- ‘/api/health‘
- ‘/api/project-info‘
- ‘/api/predict‘
- ‘/api/reload‘

Waitress

Why it is used:

- Flask’s built-in server is for development
- Waitress is used for production serving

How it works here:

- when deployed, Waitress runs the Flask app as a proper production server

JSON

Why it is used:

- JSON is simple and standard for frontend-backend communication

How it works here:

- frontend sends user text as JSON
- backend returns predictions and status as JSON

7. Machine Learning Tools Used

TensorFlow / Keras

Why it is used:

- TensorFlow and Keras are used to build and train the LSTM model
- they are standard tools for deep learning

How it works here:

- tokenizer converts words into numbers
- padded sequences are created
- an embedding layer learns word representations
- an LSTM layer learns sequence patterns
- a dense layer predicts the probability of the next word

Main training file:

- ‘training/train_lstm.py‘

Tokenizer

Why it is used:

- a machine learning model cannot understand raw words directly
- words must be converted into numeric IDs

How it works here:

- builds a vocabulary from the dataset
- converts sentences into sequences of integers
- the same tokenizer is saved and reused during prediction

pad_sequences

Why it is used:

- machine learning inputs should usually have the same length

How it works here:

- shorter sequences are padded so all inputs have a consistent size

Embedding Layer

Why it is used:

- raw word IDs are not meaningful enough
- embedding converts them into dense vector representations

How it works here:

- helps the model learn relationships between words

LSTM Layer

Why it is used:

- LSTM is designed for sequence data
- text depends on word order, so sequence learning matters

How it works here:

- it learns how previous words affect the next word
- it is the main deep learning part of the project

Dense Layer With Softmax

Why it is used:

- the final layer must choose the most likely next word from the vocabulary

How it works here:

- outputs probabilities for many words
- top words are selected from these probabilities

NumPy

Why it is used:

- NumPy is useful for array handling and efficient numeric operations

How it works here:

- manages training arrays
- helps sort and process prediction scores

scikit-learn

Why it is used:

- it provides useful utilities for splitting data

How it works here:

- ‘train_test_split‘ is used to create training and validation sets
- this gives a more honest evaluation than using only training accuracy

8. Dataset And How It Is Connected

Current dataset file:

- ‘data/sample_corpus.txt‘

What the dataset contains:

- example text lines related to next word prediction and project presentation

How the dataset is connected to the project:

1. The dataset text file is stored in ‘data/‘.
2. The training script reads that text file.
3. Text is cleaned and split into sentence chunks.
4. The tokenizer learns the vocabulary from that dataset.
5. Training sequences are generated from the tokenized text.
6. The model learns which words commonly come next.
7. The tokenizer and model are saved in ‘artifacts/‘.
8. During app runtime, the backend loads those saved files.
9. User input is converted with the same tokenizer.
10. The backend predicts next words using the trained model.

Why this connection is important:

- the model and tokenizer must match the same dataset vocabulary
- if the dataset changes, the model should be retrained
- a larger and better dataset usually gives better predictions

9. Fallback Predictor

Why it is used:

- the app should still work even before the LSTM model is trained

How it works here:

- a statistical n-gram fallback predictor is used
- it reads the sample corpus and builds unigram, bigram, and trigram relationships
- if LSTM files are missing, the backend still returns suggestions

File used:

- ‘backend/ngram_fallback.py‘

Why this is useful:

- the project remains demo-ready
- frontend and backend can be tested immediately

10. How Frontend And Backend Are Connected

The connection works like this:

1. User types text in the frontend.
2. JavaScript sends a POST request to ‘/api/predict‘.
3. Flask receives the request.
4. The backend predictor processes the text.
5. Prediction results are returned as JSON.
6. JavaScript receives the response.
7. Suggestions are shown on the screen.

This is the most important connection in the project.

Example:

- frontend sends: ‘{ "text": "machine learning", "top_n": 3 }‘
- backend returns: predicted next words with scores and source

11. How Backend And Model Are Connected

The backend does not train the model directly during user requests.

Instead, it works like this:

1. Training is done separately in ‘training/train_lstm.py‘.
2. The trained model is saved in ‘artifacts/next_word_model.keras‘.
3. The tokenizer is saved in ‘artifacts/tokenizer.pkl‘.
4. Metadata is saved in ‘artifacts/metadata.json‘.
5. Metrics are saved in ‘artifacts/metrics.json‘.
6. The backend loads these files when starting or reloading.

This separation is good because:

- training is slow and should not happen during user requests
- prediction should be fast
- the architecture stays clean and modular

12. Why The Updated Training Pipeline Is Better

Problems in the original report code:

- tokenizer saving logic was incorrect
- validation was missing
- model loss settings were inconsistent
- training quality was hard to trust

Improvements in the updated training pipeline:

- proper validation split is used
- correct sparse categorical loss is used
- early stopping is used
- learning rate reduction is used
- metrics, metadata, and training history are saved
- preview predictions are generated

This makes the project more correct and more professional.

13. Testing Tools Used

Flask Test Client

Why it is used:

- it allows quick route testing without manually opening the browser

How it works here:

- ‘tests/smoke_test.py‘ calls the main routes and checks they return success

Routes tested:

- ‘/‘
- ‘/api/health‘
- ‘/api/project-info‘
- ‘/api/predict‘

Why it is useful:

- helps confirm that frontend serving and API routes are working

14. Deployment Tools Used

Docker

Why it is used:

- Docker packages the project into a container
- this makes deployment easier and more consistent

How it works here:

- the Dockerfile installs dependencies
- copies project files
- starts the app using Waitress

File used:

- ‘Dockerfile‘

PowerShell Helper Scripts

Files:

- ‘run_local.ps1‘
- ‘train_model.ps1‘

Why they are used:

- make common commands easier to run

How they work:

- ‘run_local.ps1‘ starts the app
- ‘train_model.ps1‘ runs the training script

15. How To Run The Project

Basic app run steps:

1. Open PowerShell in the project folder.
2. Create a virtual environment.
3. Activate it.
4. Install ‘requirements.txt‘.
5. Run ‘python app.py‘.
6. Open ‘http://localhost:5000‘.

16. How To Train The Real Model

Training steps:

1. Use Python 3.11.
2. Create a fresh virtual environment.
3. Install ‘requirements-lstm.txt‘.
4. Place your dataset file inside ‘data/‘.
5. Run the training script.
6. Check whether files are created inside ‘artifacts/‘.
7. Start the app again.

After that, the app should switch from fallback mode to LSTM mode automatically.

17. What To Replace For A Real College Project

To make the project more realistic and final: